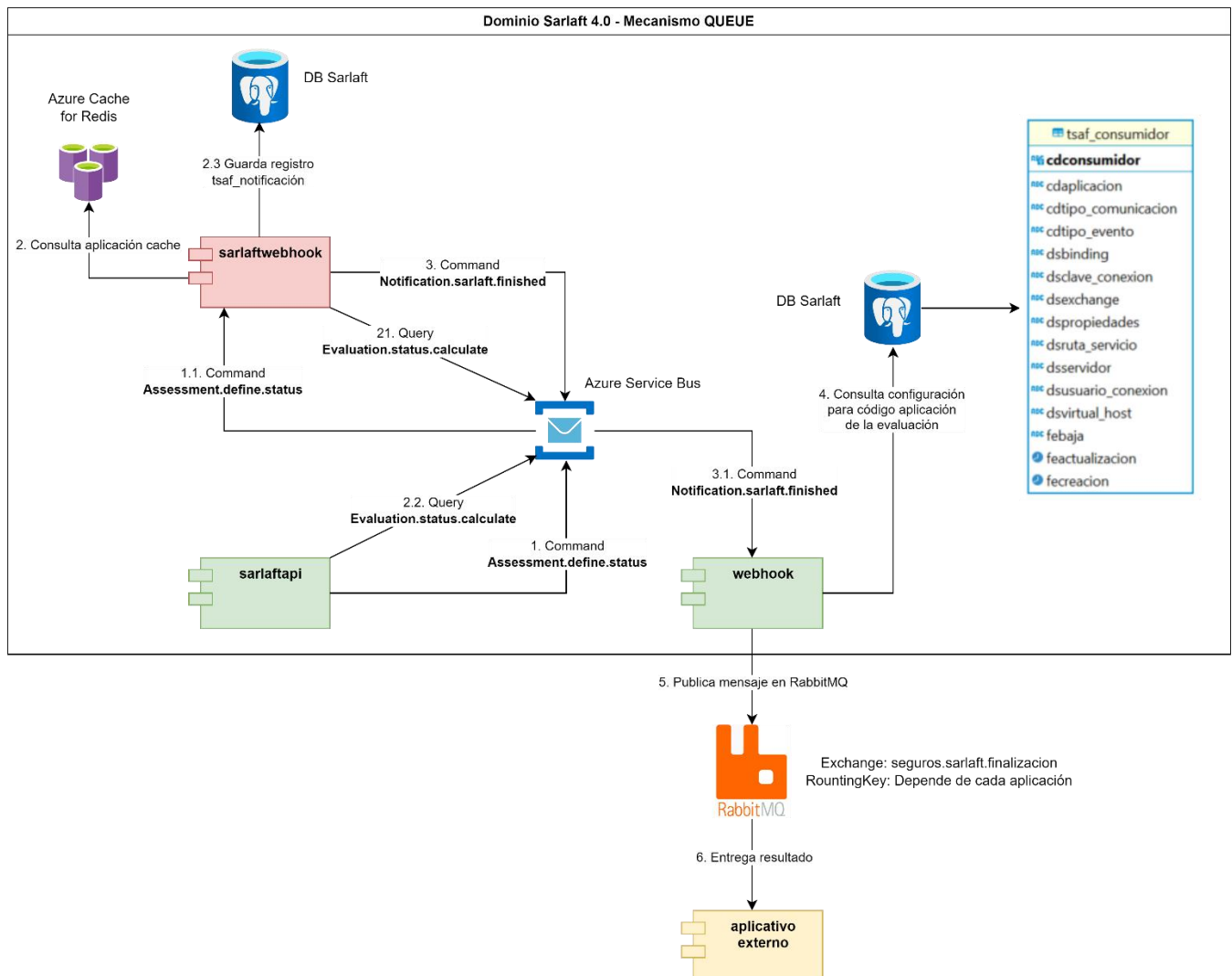


Diagrama de la arquitectura actual del proceso webhook en Sarlaft 4.0

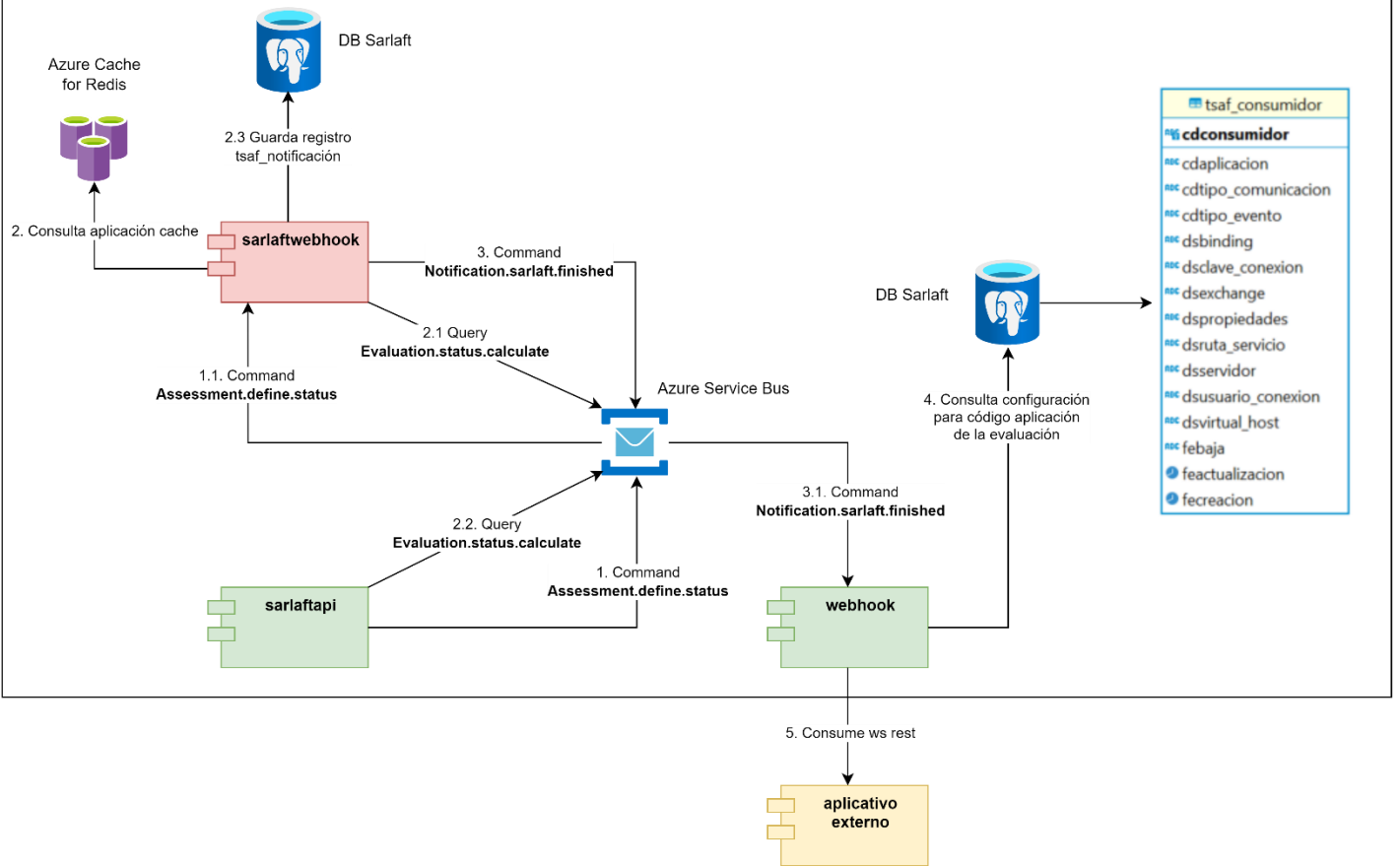
Webhook de finalización

Comunicación a través del mecanismo de QUEUE y REST:

En este esquema, el microservicio sarlaftwebhook se encarga de recibir el comando "Assessment.define.status" y ejecutar el caso de uso AssessStatusUseCase#assess. Este caso de uso busca la información de la aplicación en el Azure Cache de Redis o, en su defecto, en la base de datos del aplicativo sarlaft 4.0 si no la encuentra en caché, con el fin de determinar si se deben aplicar notificaciones. En caso de que la aplicación esté parametrizada para recibir notificaciones, se procede a obtener el estado de la evaluación consultándola en la base de datos de sarlaft 4.0. Se valida que el estado de la evaluación corresponda a alguno de los siguientes estados: FINALIZADO, RECHAZADO, PENDIENTE_ACCION_MANUAL, FINALIZADO_EXCEPCION, FINALIZADO_SIN_CARGA. Si el estado de la evaluación coincide con alguno de los mencionados anteriormente, se registra un evento en la tabla tsaf_notificacion y se lanza el comando "Notification.sarlaft.finished", el cual es recibido por el microservicio integrador webhook.



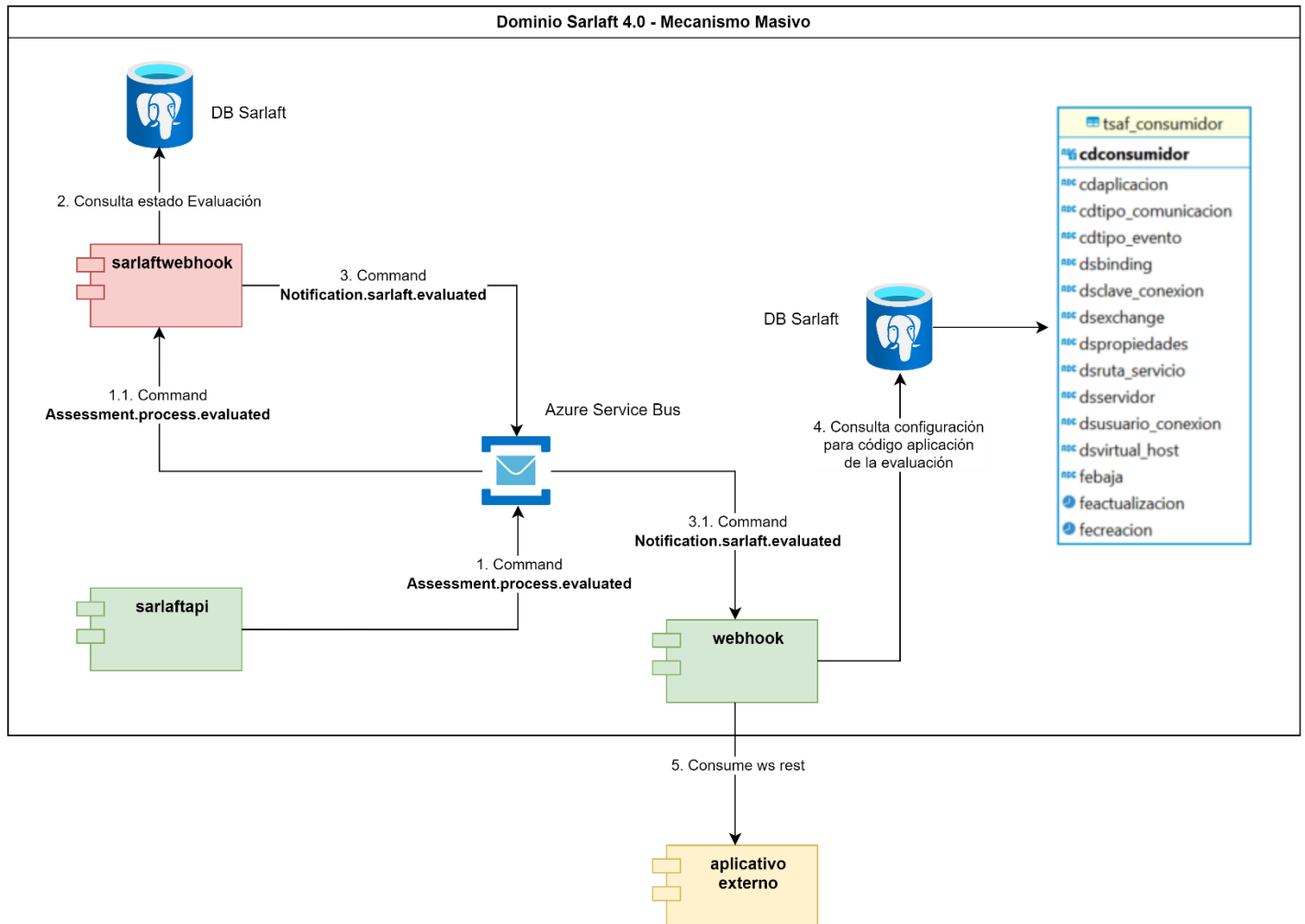
Dominio Sarlaft 4.0 - Mecanismo REST



Webhook de evaluación

Comunicación a través del mecanismo masivo:

En este esquema, el microservicio sarlaftwebhook se encarga de recibir el comando "Assessment.process.evaluated" y ejecutar el caso de uso AssessProcessUseCase#processAssess. Este caso de uso consulta el estado de cada una de las evaluaciones recibidas en la base de datos de sarlaft 4.0 y, finalmente, emite el comando "Notification.sarlaft.evaluated", el cual es recibido por el microservicio integrador webhook.



PROPUESTA 1

OMITIR LA COMUNICACIÓN QUE PASA POR EL AZURE SERVICE BUS

En esta primera propuesta se busca eliminar la comunicación del Azure Service Bus que se tiene para comunicar el microservicio de sarlaftapi con el microservicio de sarlaftwebhook a través de comandos y queries.

Diseño del nuevo proceso de webhook, interacción de componentes, cambio de comunicaciones de mensajería tipo Query a comunicación por servicio rest.

Webhook de finalización

Comunicación a través del mecanismo de QUEUE y REST:

Para este mecanismo al eliminar el microservicio sarlaftwebhook, se propone:

- Eliminar del microservicio sarlaftapi la comunicación por el comando "Assessment.define.status"
- Implementar en el microservicio sarlaftapi un caso de uso llamado AssessStatusUseCase#assess como el que se viene manejando en el microservicio de sarlaftwebhook con el fin de que ahora sea sarlaftapi el que se encargue de buscar la información de la aplicación en el Azure Cache de Redis o, en su defecto, en la base de datos del aplicativo sarlaft 4.0 si no la encuentra en caché, con el fin de determinar si se deben aplicar notificaciones. En caso de que la aplicación esté parametrizada para recibir notificaciones, se valida que el estado de la evaluación que se encuentra en flujo corresponda a alguno de los siguientes estados: FINALIZADO, RECHAZADO, PENDIENTE_ACCION_MANUAL, FINALIZADO_EXCEPCION, FINALIZADO_SIN_CARGA. Si el estado de la evaluación coincide con alguno de los mencionados anteriormente, se registra un evento en la tabla tsaf_notificacion y se lanza el comando "Notification.sarlaft.finished", el cual es recibido por el microservicio integrador webhook.
- Implementar el comando "Notification.sarlaft.finished" en el microservicio sarlaftapi (como actualmente se maneja en sarlaftwebhook) para enviarse desde sarlaftapi y así comunicarse con el microservicio integrador sarlaftwebhook mi.
- Se eliminaría la comunicación por query para "Evaluation.status.calculate" ya que se tomaría directamente desde sarlaftapi donde está implementado.
- Debido a que ya no se utilizaría el comando "Assessment.define.status" se debe cambiar la comunicación para que ahora se llame el nuevo caso de uso "AssessStatusUseCase#assess" en vez del comando en las siguientes clases:

sura.sarlaft4.usecase.assessment.AgregarEvidenciaUseCase#enviarNotificacionEstado

sura.sarlaft4.usecase.assessment.validacionidentidad.ActualizarEvidencialdentidadUseCase#actualizarEvidencias
Refactor

sura.sarlaft4.usecase.assessment.validacionregistraduria.ActualizarEvidenciaRegistraduriaUseCase#actualizarEvidencia

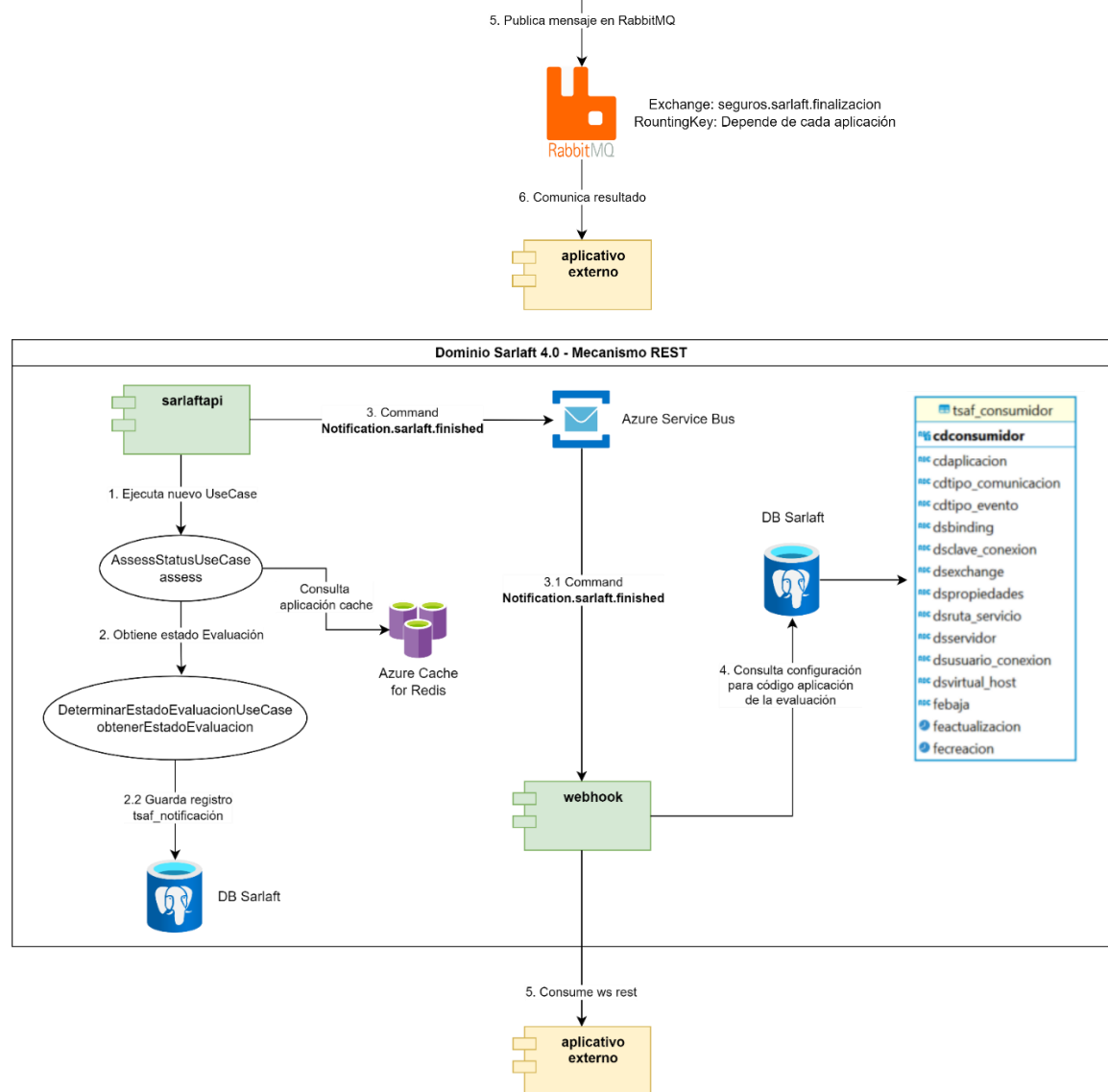
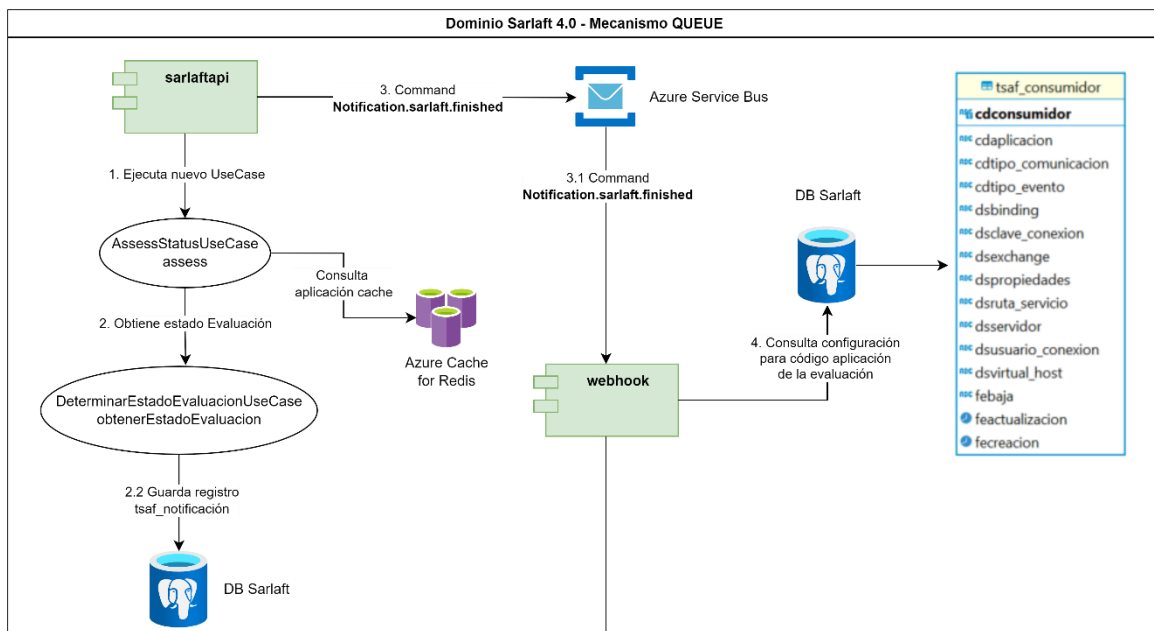
sura.sarlaft4.usecase.sarlaft.GuardarDocumentoUseCase#procesarRequisito

sura.sarlaft4.usecase.sarlaft.GuardarFormularioUseCase#guardar

sura.sarlaft4.usecase.sarlaft.OmitirFormularioUseCase#procesar

sura.sarlaft4.usecase.terceros.GuardarFormularioTercerosUseCase#guardarFormularioTerceros

sura.sarlaft4.usecase.util.notificacion.EnviaNotificacionEstadoUtil#enviarNotificacionEstado

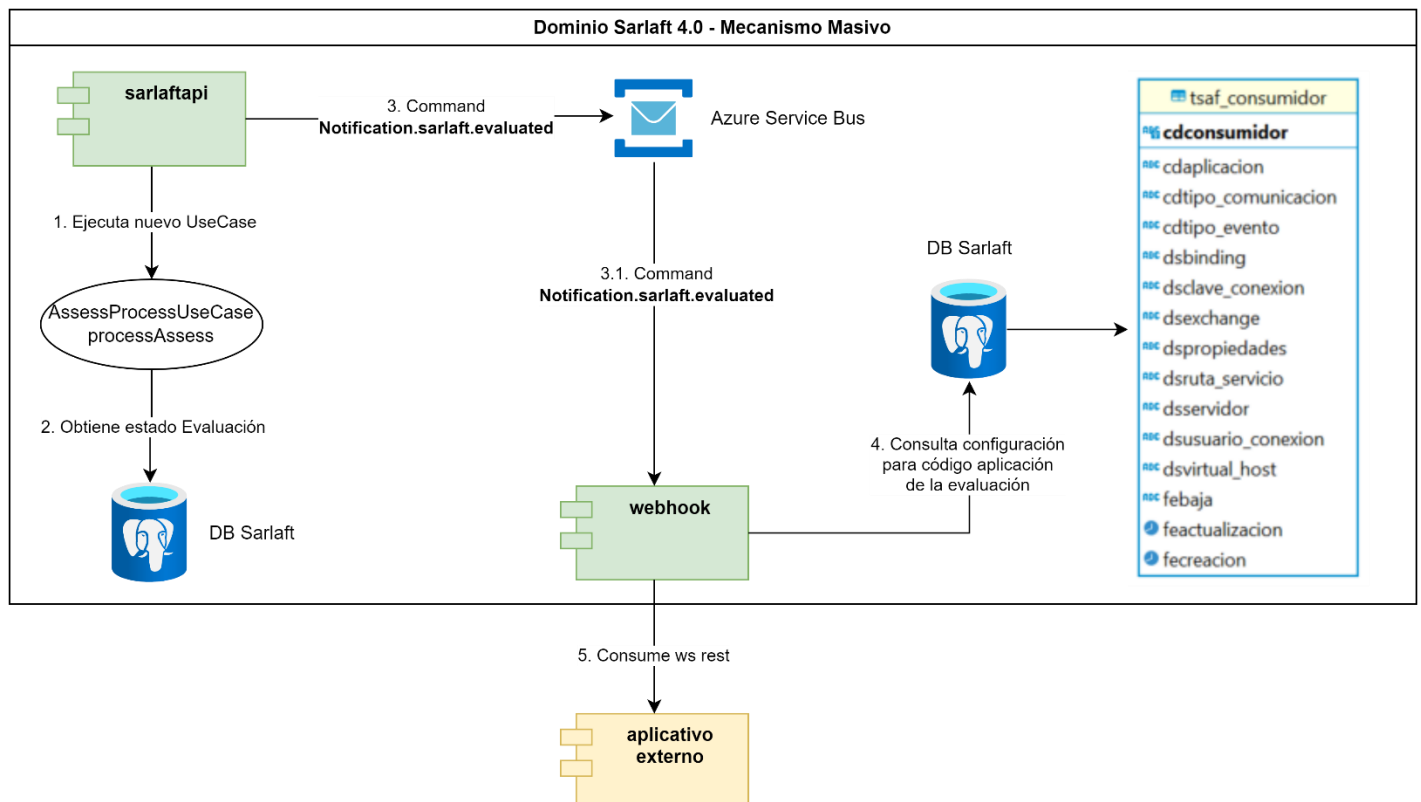


Webhook de evaluación

Comunicación a través del mecanismo masivo:

Para este mecanismo al eliminar el microservicio sarlaftwebhook, se propone:

- Se eliminaría del microservicio sarlaftapi la comunicación por comando "Assessment.process.evaluated" en `sura.sarlaft4.reactive.adapter.notificacion.NotificacionAsynpter`.
- Implementar en el microservicio sarlaftapi el caso de uso `AssessProcessUseCase#processAssess`, similar al que se maneja en el microservicio sarlaftwebhook. Este caso de uso se encarga de consultar el estado de cada una de las evaluaciones recibidas en la lista de evaluaciones del objeto `EvaluacionMasiva` en la base de datos de sarlaft 4.0. Finalmente, construye el objeto `NotificacionEvaluacionNegocio` el cual se lanza por el comando "Notification.sarlaft.evaluated", que es recibido por el microservicio integrador webhook.
- Implementar el comando "Notification.sarlaft.evaluated" para enviarse desde sarlaftapi y así comunicarse con el microservicio sarlaftwebhook mi.
- Debido a que ya no se utilizaría el comando "Assessment.process.evaluated" se debe cambiar la comunicación para que ahora se llame el nuevo caso de uso "AssessProcessUseCase#processAssess" en vez del comando en las siguientes clases:
`sura.sarlaft4.usecase.assessment.ProcesarMasivosUseCase#enviarCommando`



Identificación de los puntos a mejorar y riesgos con la aplicación del cambio

Mejoras:

- Eliminar el punto de fallo del microservicio de sarlaftwebhook.
- Eliminar el cuello de botella de atender el query por parte del microservicio de sarlaftapi.
- Al evitar la comunicación por el azure service bus se mejoran los siguientes ítems:
 - Gastos operativos.
 - Tarifas de Transferencia de Datos.
 - Retardo en la Entrega de los mensajes.
 - Dependencia con la Red cuando se presente conectividad limitada o inestable.
- Al evitar la comunicación por el azure service bus se elimina el problema con el tamaño máximo de los mensajes.
- Los cambios realizados en el microservicio de sarlaftapi no requerirían pruebas de seguridad dinámicas lo cual elimina la espera que implica que sean programadas.

Riesgos y contras:

- Transferir la responsabilidad que venía manejando el microservicio sarlaftwebhook al microservicio sarlaftapi.
- Se deben realizar varios cambios a nivel de programación en el microservicio sarlaftapi.
- Aumenta la complejidad accidental en el microservicio de sarlaftapi.
- Los cambios que se deben realizar en el microservicio sarlaftapi aplica para pruebas de seguridad estáticas por checkmarx.
- Al eliminar de la comunicación el paso de los mensajes a través del azure service bus se perdería:
 - Gestión de los mensajes en el dead letter queue
 - Trazabilidad de los mensajes y eventos
 - Gestión de los mensajes: consulta, filtrado, estadísticas
 - Garantía de Entrega de los mensajes
 - Persistencia de Mensajes: los mensajes se almacenan de manera duradera, asegurando que no se pierdan y puedan ser recuperados y procesados posteriormente.
 - Autoescalado: manejo automático del aumento de la carga de trabajo sin intervención manual.
- Al trasladarse la responsabilidad que tiene actualmente el microservicio sarlaftwebhook (pensando en ser eliminado en un futuro) al microservicio sarlaftapi, podría aumentar los tiempos de las transacciones sobre todo con los servicios de assessment y guardar formulario.

Identificar las HUs técnicas necesarias para aplicar el cambio en el aplicativo de sarlaft 4.0

1. HU – Sarlaft4 – Eliminar comando Assessment.define.status del microservicio de sarlaftapi

Descripción: Con el fin de cortar la comunicación entre el microservicio sarlaftapi – Azure Service Bus – microservicio sarlaftwebhook, se eliminará el comando Assessment.define.status del microservicio sarlaftapi, como consecuencia de sebe implementar en el microservicio sarlaftapi un caso de uso llamado AssessStatusUseCase#assess como el que se viene manejando en el microservicio de sarlaftwebhook con el fin de que ahora sea sarlaftapi el que se encargue de buscar la información de la aplicación en el Azure Cache de Redis o, en su defecto, en la base de datos del aplicativo sarlaft 4.0 si no la encuentra en caché, con el fin de determinar si se deben aplicar notificaciones. En caso de que la aplicación esté parametrizada para recibir notificaciones, se valida que el estado de la evaluación que se encuentra en flujo corresponda a alguno de los siguientes estados: FINALIZADO, RECHAZADO, PENDIENTE_ACCION_MANUAL, FINALIZADO_EXCEPCION, FINALIZADO_SIN_CARGA. Si el estado de la evaluación coincide con alguno de los mencionados anteriormente, se registra un evento en la tabla tsaf_notificacion y se lanza el comando "Notification.sarlaft.finished", el cual es recibido por el microservicio integrador webhook. Debido a que ya no se utilizaría el comando "Assessment.define.status" se debe cambiar la comunicación para que ahora se llame el nuevo caso de uso "AssessStatusUseCase#assess" en vez del comando en las siguientes clases:

sura.sarlaft4.usecase.assessment.AgregarEvidenciaUseCase#enviarNotificacionEstado

sura.sarlaft4.usecase.assessment.validacionidentidad.ActualizarEvidencialIdentidadUseCase#actualizarEvidencias
Refactor

sura.sarlaft4.usecase.assessment.validacionregistraduria.ActualizarEvidenciaRegistraduriaUseCase#actualizarEvidencia

sura.sarlaft4.usecase.sarlaft.GuardarDocumentoUseCase#procesarRequisito

sura.sarlaft4.usecase.sarlaft.GuardarFormularioUseCase#guardar

sura.sarlaft4.usecase.sarlaft.OmitirFormularioUseCase#procesar

sura.sarlaft4.usecase.terceros.GuardarFormularioTercerosUseCase#guardarFormularioTerceros

sura.sarlaft4.usecase.util.notificacion.EnviaNotificacionEstadoUtil#enviarNotificacionEstado

2. HU – Sarlaft4 – Implementar el comando Notification.sarlaft.finished en el microservicio sarlaftapi

Descripción: Implementar el comando Notification.sarlaft.finished en el microservicio sarlaftapi (como actualmente se maneja en sarlaftwebhook) para enviarse desde sarlaftapi y así comunicarse con el microservicio integrador sarlaftwebhook mi.

3. HU – Sarlaft4 – Eliminar comando Assessment.process.evaluated del microservicio de sarlaftapi

- **Descripción:** Con el fin de cortar la comunicación entre el microservicio sarlaftapi – azure service bus – microservicio sarlaftwebhook, se eliminará el comando Assessment.process.evaluated del microservicio sarlaftapi, como consecuencia de sebe implementar en el microservicio sarlaftapi un caso de uso llamado AssessProcessUseCase#processAssess similar al que se maneja en el microservicio sarlaftwebhook. Este caso de uso se encarga de consultar el estado de cada una de las evaluaciones recibidas en la lista de evaluaciones del objeto EvaluacionMasiva en la base de datos de sarlaft 4.0. Finalmente, construye el objeto NotificacionEvaluacionNegocio el cual se lanza por el comando "Notification.sarlaft.evaluated", que es recibido por el microservicio integrador webhook. Debido a que ya no se utilizaría el comando

"Assessment.process.evaluated" se debe cambiar la comunicación para que ahora se llame el nuevo caso de uso "AssessProcessUseCase#processAssess" en vez del comando en las siguientes clases:
sura.sarlaft4.usecase.assessment.ProcesarMasivosUseCase#enviarCommando

4. HU – Sarlaft4 – Implementar el comando Notification.sarlaft.evaluated en el microservicio sarlaftapi

Descripción: Implementar el comando Notification.sarlaft.evaluated en el microservicio sarlaftapi (como actualmente se maneja en sarlaftwebhook) para enviarse desde sarlaftapi y así comunicarse con el microservicio integrador sarlaftwebhook mi.

5. HU – Sarlaft4 – Gestión para apagado del microservicio sarlaftwebhook.

Descripción: Esta historia es para llevar a cabo todo lo que implica la gestión para el apagado del microservicio sarlaftwebhook.

PROPUESTA 2

MANTENER LA COMUNICACIÓN QUE PASA POR EL AZURE SERVICE BUS

En esta segunda propuesta se busca mantener la comunicación a través del azure service bus, lanzando comandos provenientes de sarlaftapi, pasando por el azure service bus y que escucharía nuevamente el microservicio sarlaftapi.

Diseño del nuevo proceso de webhook, interacción de componentes, cambio de comunicaciones de mensajería tipo Query a comunicación por servicio rest.

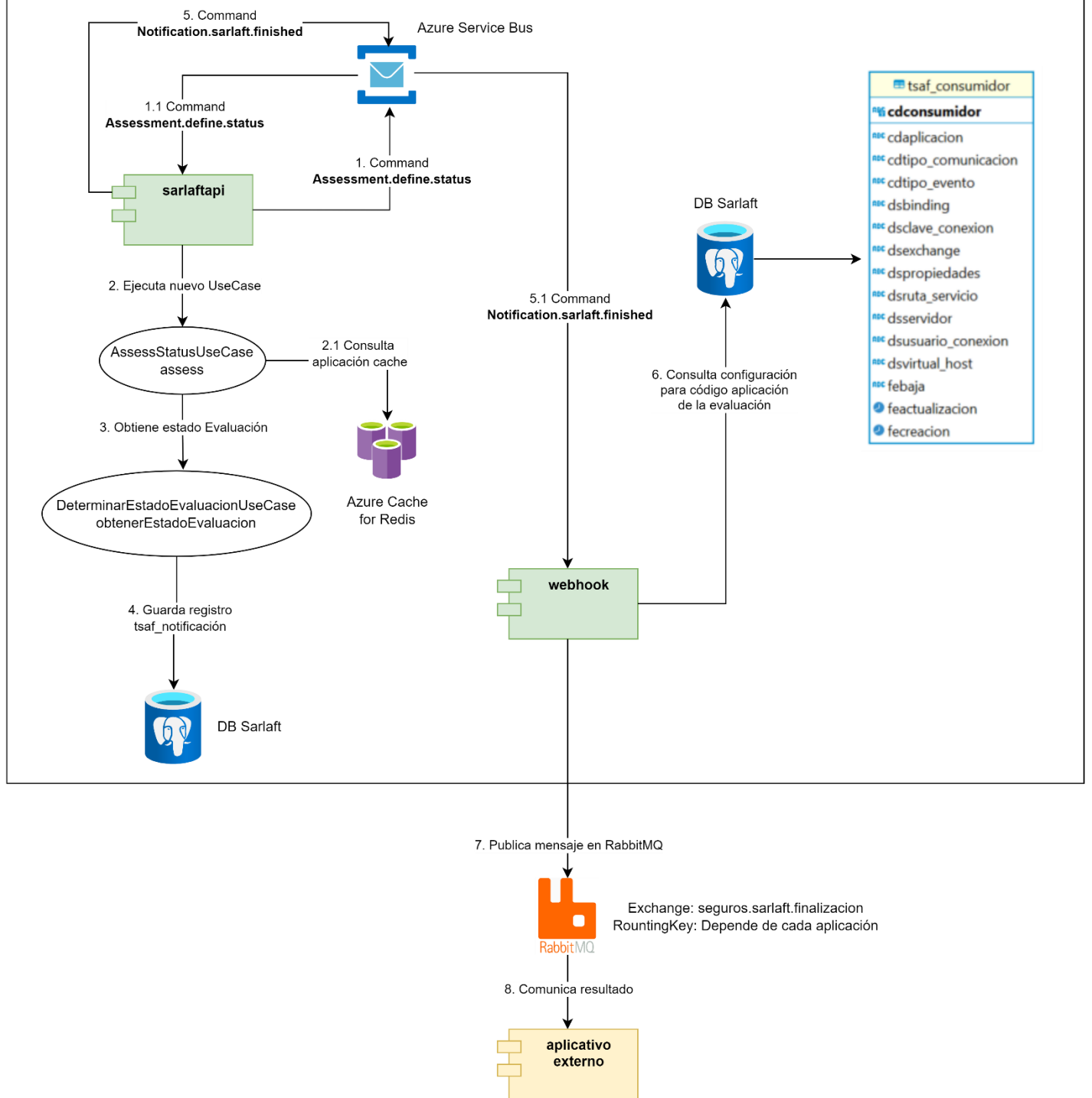
Webhook de finalización

Comunicación a través del mecanismo de QUEUE y REST:

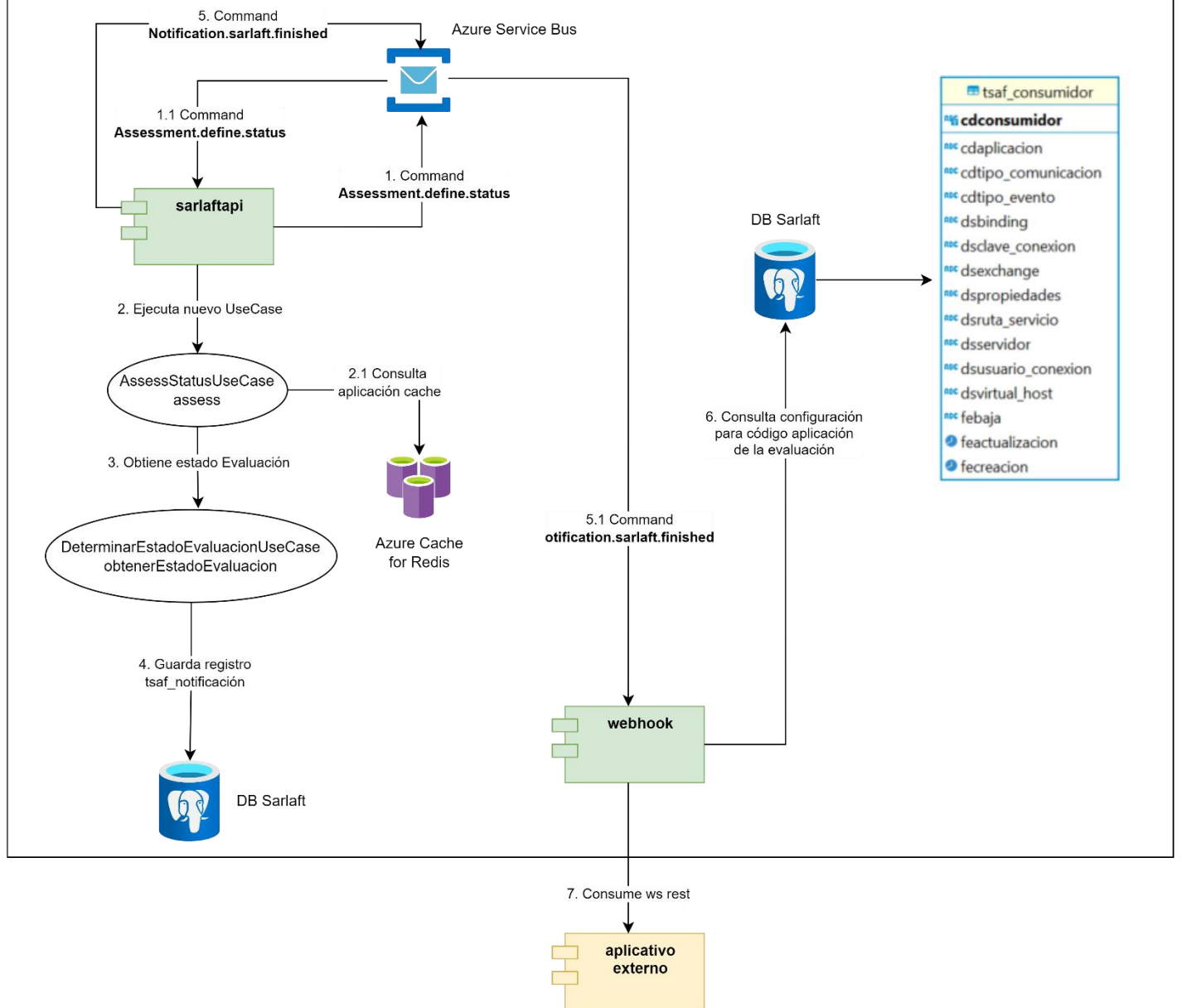
Para este mecanismo al eliminar el microservicio sarlaftwebhook, se propone:

- Mantener en el microservicio sarlaftapi la comunicación por el comando "Assessment.define.status".
- Implementar en el microservicio sarlaftapi que escuche el comando "Assessment.define.status".
- Implementar en el microservicio sarlaftapi un caso de uso llamado AssessStatusUseCase#assess como el que se viene manejando en el microservicio de sarlaftwebhook con el fin de que ahora sea sarlaftapi el que se encargue de buscar la información de la aplicación en el Azure Cache de Redis o, en su defecto, en la base de datos del aplicativo sarlaft 4.0 si no la encuentra en caché, con el fin de determinar si se deben aplicar notificaciones. En caso de que la aplicación esté parametrizada para recibir notificaciones, se valida que el estado de la evaluación que se encuentra en flujo corresponda a alguno de los siguientes estados: FINALIZADO, RECHAZADO, PENDIENTE_ACCION_MANUAL, FINALIZADO_EXCEPCION, FINALIZADO_SIN_CARGA. Si el estado de la evaluación coincide con alguno de los mencionados anteriormente, se registra un evento en la tabla tsaf_notificacion y se lanza el comando "Notification.sarlaft.finished", el cual es recibido por el microservicio integrador webhook.
- Implementar el comando "Notification.sarlaft.finished" en el microservicio sarlaftapi (como actualmente se maneja en sarlaftwebhook) para enviarse desde sarlaftapi y así comunicarse con el microservicio integrador sarlaftwebhook mi.
- Se eliminaría la comunicación por query para "Evaluation.status.calculate" ya que se tomaría directamente desde sarlaftapi donde está implementado.

Dominio Sarlaft 4.0 - Mecanismo QUEUE



Dominio Sarlaft 4.0 - Mecanismo REST

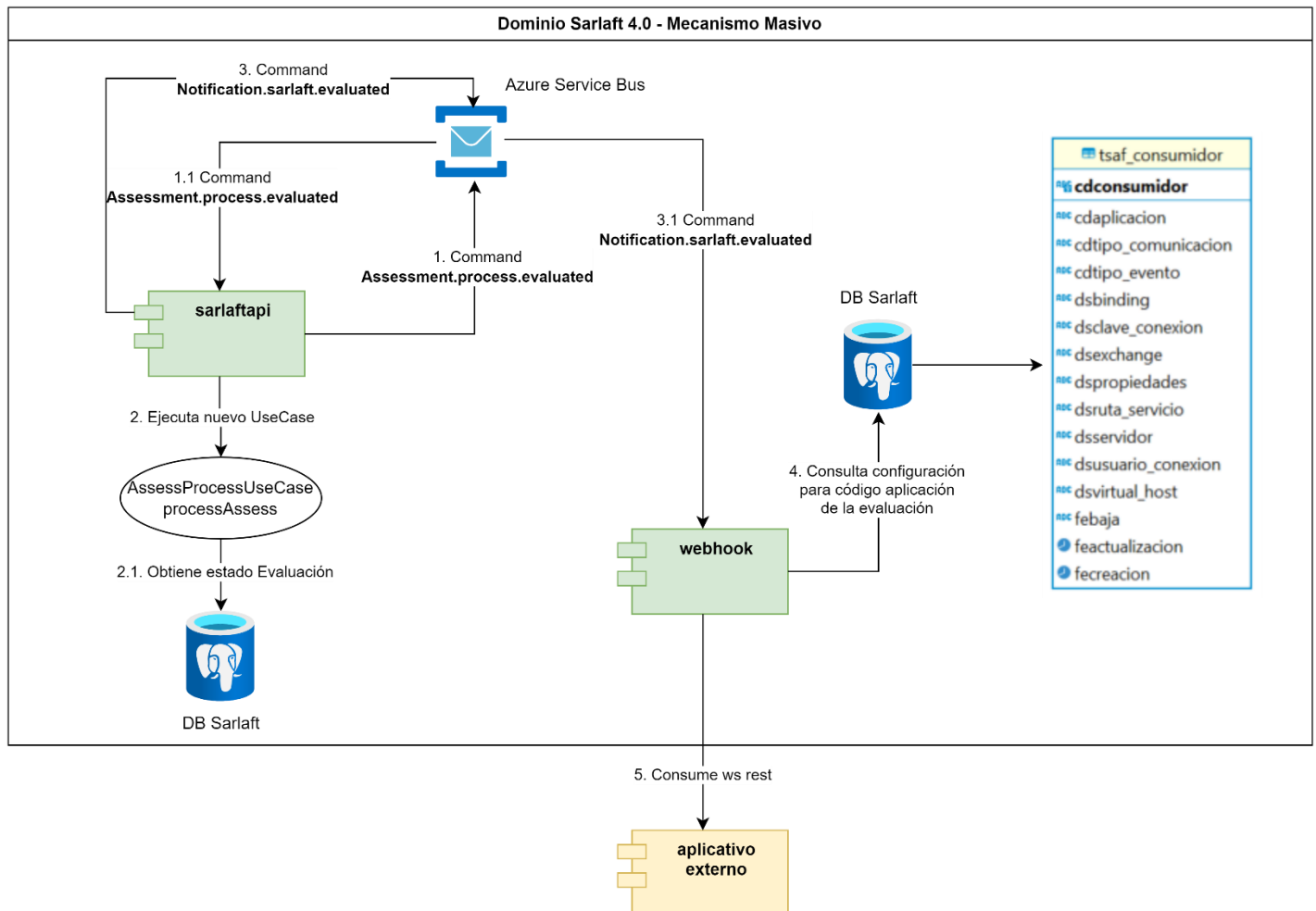


Webhook de evaluación

Comunicación a través del mecanismo masivo:

Para este mecanismo al eliminar el microservicio sarlaftwebhook, se propone:

- Mantener la comunicación por comando "Assessment.process.evaluated" en `sura.sarlaft4.reactive.adapter.notificacion.NotificacionAsynpter`.
- Implementar en el microservicio sarlaftapi que escuche el comando "Assessment.process.evaluated".
- Implementar en el microservicio sarlaftapi el caso de uso `AssessProcessUseCase#processAssess`, similar al que se maneja en el microservicio sarlaftwebhook. Este caso de uso se encarga de consultar el estado de cada una de las evaluaciones recibidas en la lista de evaluaciones del objeto `EvaluacionMasiva` en la base de datos de sarlaft 4.0. Finalmente, construye el objeto `NotificacionEvaluacionNegocio` el cual se lanza por el comando "Notification.sarlaft.evaluated", que es recibido por el microservicio integrador webhook.
- Implementar el comando "Notification.sarlaft.evaluated" para enviarse desde sarlaftapi y así comunicarse con el microservicio sarlaftwebhook mi.



Identificación de los puntos a mejorar y riesgos con la aplicación del cambio

Mejoras:

- Eliminar el punto de fallo del microservicio de sarlaftwebhook.
- Eliminar el cuello de botella de atender el query por parte del microservicio de sarlaftapi.
- Al mantener la comunicación por el Azure Service Bus se conservaría:
 - Gestión de los mensajes en el dead letter queue.
 - Trazabilidad de los mensajes y eventos.
 - Gestión de los mensajes: consulta, filtrado, estadísticas.
 - Garantía de Entrega de los mensajes.
 - Persistencia de Mensajes: los mensajes se almacenan de manera duradera, asegurando que no se pierdan y puedan ser recuperados y procesados posteriormente.
 - Autoescalado: manejo automático del aumento de la carga de trabajo sin intervención manual.
- Los cambios realizados en el microservicio de sarlaftapi no requerirían pruebas de seguridad dinámicas lo cual elimina la espera que implica que sean programadas.

Riesgos y contras:

- Transferir la responsabilidad que venía manejando el microservicio sarlaftwebhook al microservicio sarlaftapi.
- Se deben realizar varios cambios a nivel de programación en el microservicio sarlaftapi.
- Aumenta la complejidad accidental en el microservicio de sarlaftapi.
- Los cambios que se deben realizar en el microservicio sarlaftapi aplica para pruebas de seguridad estáticas por checkmarx.
- Al mantener la comunicación a través del Azure Service Bus se mantendría:
 - Gastos operativos
 - Tarifas de Transferencia de Datos
 - Retardo en la Entrega de los mensajes
 - Dependencia con la Red cuando se presente conectividad limitada o inestable
- Al mantener la comunicación por el Azure Service Bus se podría presentar el problema con el tamaño máximo de los mensajes.
- Debido a que el microservicio sarlaftapi emitirá comandos que a su vez serán recibidos por él y debido a que ahora en sarlaftapi asumirá toda la transaccionalidad que se venía manejando en el microservicio sarlaftwebhook, se pronostica un encolamiento de mensajes en el Azure Service Bus.

Identificar las HUs técnicas necesarias para aplicar el cambio en el aplicativo de saralft 4.0

1. HU – Sarlaft4 – Implementar la escucha del comando `Assessment.define.status` en el microservicio de sarlaftapi

Descripción: Se debe implementar la escucha del comando `Assessment.define.status` en el microservicio sarlaftapi e implementar en el microservicio sarlaftapi un caso de uso llamado `AssessStatusUseCase#assess` como el que se viene manejando en el microservicio de sarlaftwebhook con el fin de que ahora sea sarlaftapi el que se encargue de buscar la información de la aplicación en el Azure Cache de Redis o, en su defecto, en la base de datos del aplicativo sarlaft 4.0 si no la encuentra en caché, con el fin de determinar si se deben aplicar notificaciones. En caso de que la aplicación esté parametrizada para recibir notificaciones, se valida que el estado de la evaluación que se encuentra en flujo corresponda a alguno de los siguientes estados: FINALIZADO, RECHAZADO, PENDIENTE_ACCION_MANUAL, FINALIZADO_EXCEPCION, FINALIZADO_SIN_CARGA. Si el estado de la evaluación coincide con alguno de los mencionados anteriormente, se registra un evento en la tabla tsaf_notificacion y se lanza el comando "Notification.sarlaft.finished", el cual es recibido por el microservicio integrador webhook.

2. HU – Sarlaft4 – Implementar el comando `Notification.sarlaft.finished` en el microservicio sarlaftapi

Descripción: Implementar el comando `Notification.sarlaft.finished` en el microservicio sarlaftapi (como actualmente se maneja en sarlaftwebhook) para enviarse desde sarlaftapi y así comunicarse con el microservicio integrador sarlaftwebhook mi.

3. HU – Sarlaft4 – Eliminar comando `Assessment.define.status` del microservicio de sarlaftwebhook

Descripción: Para cortar la comunicación entre el microservicio sarlaftapi – azure service bus – microservicio sarlaftwebhook, se eliminará el comando `Assessment.define.status` del microservicio sarlaftwebhook, sin eliminar el caso de uso correspondiente con el fin de cortar la comunicación con el microservicio sarlaftapi y el azure service bus, dado que cuando se pueda dar de baja al microservicio sarlaftwebhook, este ya no será utilizado en ningún ambiente.

4. HU – Sarlaft4 – Implementar la escucha del comando `Assessment.process.evaluated` en el microservicio de sarlaftapi

Descripción: Se debe implementar la escucha del comando `Assessment.process.evaluated` en el microservicio sarlaftapi y por consiguiente se debe implementar el caso de uso `AssessProcessUseCase#processAssess` como el que se viene manejando en el micro de sarlaftwebhook el cual se encarga de consultar el estado de cada una de las evaluaciones recibidas en la base de datos de sarlaft 4.0 y, finalmente, emite el comando "Notification.sarlaft.evaluated", el cual es recibido por el microservicio integrador webhook.

5. HU – Sarlaft4 – Implementar el comando `Notification.sarlaft.evaluated` en el microservicio sarlaftapi

Descripción: Implementar el comando `Notification.sarlaft.evaluated` en el microservicio sarlaftapi (como actualmente se maneja en sarlaftwebhook) para enviarse desde sarlaftapi y así comunicarse con el microservicio integrador sarlaftwebhook mi.

6. HU – Sarlaft4 – Eliminar comando `Assessment.process.evaluated` del microservicio de sarlaftwebhook

Descripción: Para cortar la comunicación entre el microservicio sarlaftapi – azure service bus – microservicio sarlaftwebhook, se eliminará el comando `Assessment.process.evaluated` del microservicio sarlaftwebhook, sin eliminar el caso de uso correspondiente con el fin de cortar la comunicación con el microservicio sarlaftapi y el azure service bus, dado que cuando se pueda dar de baja al microservicio sarlaftwebhook, este ya no será utilizado en ningún ambiente.

7. HU – Sarlaft4 – Gestión para apagado del microservicio sarlaftwebhook.

Descripción: Esta historia es para llevar a cabo todo lo que implica la gestión para el apagado del microservicio sarlaftwebhook.